

Flutter App Langue

Application Flutter d'apprentissage des langues basée sur un algorithme de répétition espacée (SRS).
Ce projet sert à la fois de terrain d'apprentissage pour **Dart/Flutter** et de base pour une application complète de mémorisation de vocabulaire et de révision intelligente.

Objectifs

- Créer une application Flutter multiplateformes (Android, iOS, Web, Desktop).
 - Implémenter un système de répétition espacée (Spaced Repetition System) inspiré d'Anki/SM-2.
 - Construire une architecture modulaire et extensible avec une base de données SQLite.
 - Appliquer les bonnes pratiques de conception logicielle (clean architecture, séparation des couches, code testable).
 - s'engager dans un projet en se concentrant sur la *rigueur* et la *qualité* du projet.
-

Architecture (MVP)

L'application suit une architecture modulaire, organisée autour de 4 blocs principaux :

- **UI**
Écrans Flutter (Home, Sessions, Stats, Settings).
Aucun accès direct au Domain ou à la base de données.
- **Application / Controllers**
Orchestration de la logique applicative (sessions, navigation, statistiques).
Les Controllers sont long-vivants et partagés entre les écrans.
- **Domain**
Logique métier pure :
 - contenu pédagogique (**Word** et **Sentence**)
 - sessions d'apprentissage (**Session**)
 - Objet runtime Exercice (**abstract Exercice**, **WordExercice** et **SentenceExercice**)
 - système de répétition espacée (SRS)
- **Persistence**
Accès aux données via des repositories.
Le SQL, le mapping DB ↔ Domain et les optimisations sont confinés à cette couche.

🔗 L'architecture complète est documentée dans [docs/architecture](#).

⚙️ État actuel (MVP in progress) - chronologie

- Architecture MVP définie et documentée
- Modèle SRS conçu (cf [docs/maths_srs.md](#) et [docs/hypotheses_et_info_srs.md](#))

- Séparation claire UI / Application / Domain / Persistence
- 🚧 Implémentation en cours

Le code existant est progressivement aligné sur cette architecture.